OpenVINO GenAI

Image-to-Video Support for the LTX Video Generation Pipeline

Google Summer of Code 2025 - Proposal

Applicant Information	
Full Name	Aditya Goyal
University	Manipal Institute of Technology, Karnataka, India (B.Tech, Computer and Communication Engineering)
Timezone	GMT+5:30 (IST)
Telephone	[redacted]
Email	goyaladitya2403@gmail.com
GitHub	github.com/goyaladitya05
LinkedIn	linkedin.com/in/aditya-goyal-297a2422b
Mentors	Anna Likholat, Stanislav Gonorovskii
Project Size	350 hours **Difficulty:** Medium

Submitted to the OpenVINO organization for Google Summer of Code 2025

Contents

1 About Me

1.1 Short Bio

I am Aditya Goyal, a pre-final year B.Tech student in Computer and Communication Engineering at MIT Manipal, Karnataka, India.

I never liked ML. For a long time it felt generic to me - everyone was doing it, and I wanted no part of it. About two and a half years ago, some friends pulled me into a research project on a split-second decision I almost didn't make. I had initially opted out. Over the next few months I strengthened my math, picked up the basics, and was fully dragged in before I knew what had happened.

Open-source came later, and through a more humbling route. About six months ago I interviewed for a role with a senior from a nearby college - someone I'd met on Reddit. He asked me if I had any open-source contributions. I was blank. I didn't have a great GPA and had been overlooked in the last recruitment cycle, which was very disheartening for me. After the interview I asked him about it. He said open-source is one of the best ways to learn, and that when the standard pipeline (grades, interview, job) isn't working, it's worth trying a different path. I took that seriously.

I started exploring OpenVINO in November 2025, curious about running generative AI models efficiently on Intel hardware, and partly because I had seen it while going through his GitHub contributions. He had a few PRs in OpenVINO from the previous year. By late December I had picked up my first issue #2323: GGUF LoRA adapter support and submitted a PR in mid-January 2026. It was merged on February 4. Three months later I have learned more from code review, bug traces, and maintainer feedback than I did in two years of coursework. I think he was right.

In parallel, I identified that the Keras 3 OpenVINO backend had significant operator coverage gaps. The previously opened maintainer issues had already been closed by others, so I audited the backend myself, got permission from **@rkazants** to work on parity, and expanded the scope as I went. As of March 2026, this is on track for complete parity by April 20 - well before GSoC begins, with no overlap with this project's scope.

A detailed breakdown of all contributions is in Section 5.

1.2 Relevant Open Source Work

The contributions below are directly relevant to this proposal and to why I am applying specifically for this project. Both mentors, Anna Likholat (**@likholat**) and Stanislav Gonorovskii (**@sgonorov**) have reviewed them.

- **PR #3204** MERGED GGUF LoRA Adapter Support. First PR in openvino.genai; implemented `GGUFAdapterImpl` in `src/cpp/src/lora/adapter.cpp` with `convert_gguf_name_to_hf()` for GGUF-to-HuggingFace tensor naming convention mapping.
- **PR #3362** MERGED LoRA Adapter Support for `Text2VideoPipeline`. Implemented end-to-end LoRA support for LTX-Video generation, including dynamic adapter prefix detection to support both Diffusers-trained and ComfyUI-trained community adapters.
- **PR #3431** DRAFT PR VAE Encoder for `AutoencoderKLLTXVideo`. Implements `encode()` - the component required for I2V conditioning. Python bindings, 6 tests covering output shape, seed determinism, and variation. Converted to draft after clarification from mentors that this is part of the GSoC project scope.
- **PR #3546** OPEN WWB LoRA Support for Video Generation. Extends the Who What

Benchmark (WWB) to support LoRA adapter evaluation for the Video Generation pipeline, matching existing LLM and Image Generation workflows.

1.3 Other Projects

- **Multi-Agent RL for LLM Reasoning:** Full MARL pipeline (SFT + DPO on LLaMA-3) using vLLM on an A100. Achieved **3.5% absolute accuracy gain on MATH-500** over single-agent baselines.
- **Federated Learning Research (ongoing, targeting publication):** [redacted]
- **Osiris - Vision-Based GUI Agent:** Autonomous GUI agent using YOLOv8 + MCP inspired by Microsoft OmniParser. Self-correcting loop with real-time screenshot verification.

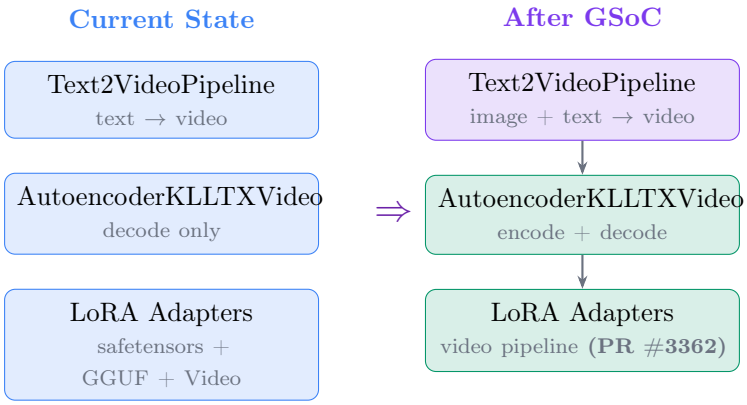
2 About The Project

2.1 Project Abstract

OpenVINO GenAI currently provides Text-to-Video generation via the LTX pipeline, which is a diffusion model that generates videos from text through iterative denoising in latent space. This project extends it with **Image-to-Video (I2V) generation**, enabling users to produce videos conditioned on an input image combined with a text prompt, running on Intel CPU and GPU. Image conditioning provides a strong visual anchor, improving control over composition and style compared to text-only generation.

This proposal covers the full I2V implementation: VAE encoder support, wiring the encoded latent into the denoising loop, full C++/Python API parity, WWB benchmarking updates, and a test suite.

2.2 Current State vs. Target State



2.3 Why This Project?

I came across this project through the GSoC listings. Having spent a few months in the OpenVINO GenAI codebase and with some prior background in diffusion models and Generative AI (I am in the early stages of a related research project), the GenAI projects naturally stood out. I started with the LoRA video generation issue (#3306), and as I worked through the codebase I became increasingly familiar with the LTX pipeline. By the time PR #3362 was under review and I had spotted the `// TODO: for img2video` comments in `AutoencoderKLLTXVideo`, this felt like a natural next step. The codebase was familiar, and the work seemed like a good continuation of what I

had already been doing.

2.4 Time Commitment

I will dedicate **40 hours per week** from May 25 to July 20 (8 weeks) during my summer break, when I have no academic commitments. From July 21, when college resumes, I will continue with **20 hours per week** for the remaining weeks through September 8.

- **High-intensity phase** (8 weeks × 40 hrs/week): 320 hours
- **Reduced phase** (7 weeks × 20 hrs/week): 140 hours
- **Total commitment** (15 weeks): **460 hours**

This comfortably exceeds the 350-hour project estimate, providing buffer for unexpected delays in review cycles, validation, or any issues that may arise. The high-intensity phase covers the most critical deliverables - I2V core integration and API parity, ensuring the project reaches a functional, reviewable state before college resumes.

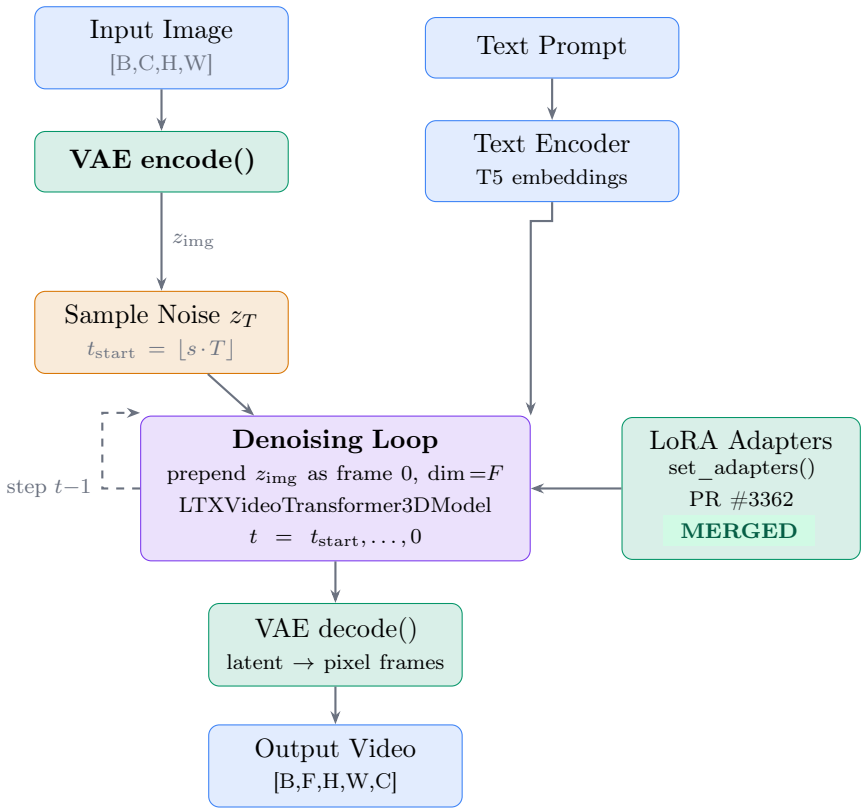
3 Technical Design

Note: After sharing this draft with the mentors, I learned that both the implementation and architecture planning for I2V support are expected to be part of the GSoC program itself. The technical design below reflects my analysis of the codebase and the Diffusers reference implementation, and should be read as a proposed starting point rather than a finalised plan. All design decisions are open for discussion and revision with the mentors during Community Bonding.

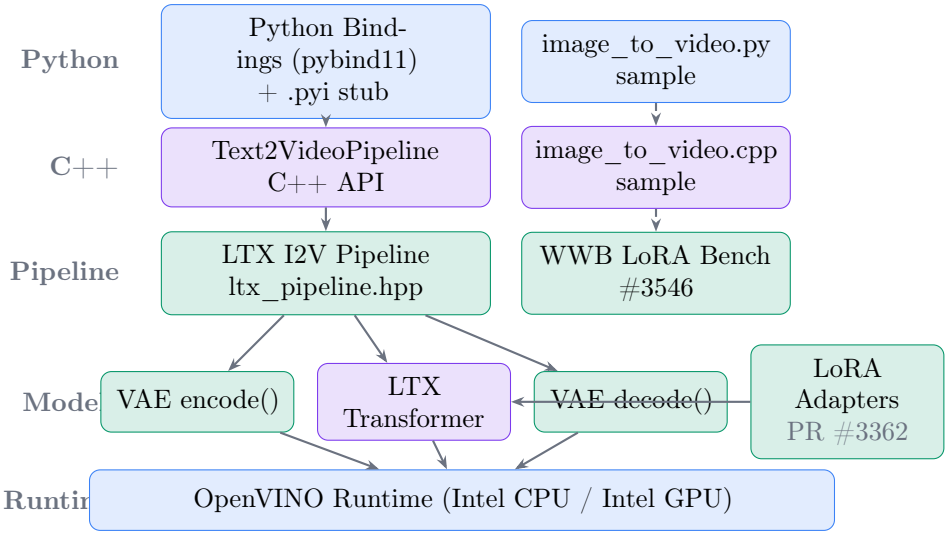
3.1 Architecture Overview

The I2V pipeline extends `Text2VideoPipeline` without introducing a separate class. This design choice keeps the API surface minimal and ensures all existing T2V functionality remains unchanged. The conditioning mechanism follows HuggingFace Diffusers' `LTXImageToVideoPipeline`: the input image is encoded into latent space *once* via `encode()`, and the resulting image latent is prepended as frame 0 of the latent tensor along the frame axis (`dim=2`). The remaining frames are initialised from noise. Frame 0 is held fixed throughout the denoising loop - the scheduler updates only frames $1 \dots F-1$, and frame 0 is re-concatenated at every step. The channel dimension is unchanged ($C = 128$ in both T2V and I2V), so no separate model export is needed. Image preprocessing reuses the existing `ImageResizer` and `ImageProcessor` classes already used by the Stable Diffusion and Flux img2img pipelines - no new utilities are needed.

3.1.1 High-Level Pipeline Flow



3.1.2 System Layers



3.2 Conditioning Mechanism

3.2.1 Mathematical Formulation

The image latent $z_{\text{img}} \in \mathbb{R}^{B\times C\times 1\times H\times W}$ is prepended to the noise latent along the frame axis to form the initial latent tensor before the denoising loop:

$$z_0 \;=\; \text{concat}\big(z_{\text{img}},\; z_{\text{noise}},\; \text{dim}=F\big) \;\in\; \mathbb{R}^{B\times C\times F\times H\times W}$$

Frame 0 carries the image conditioning and is held fixed throughout denoising. At each step t, the scheduler updates only frames $1 \ldots F-1$, and frame 0 is re-concatenated before the next step:

$$z_t \;\leftarrow\; \text{concat}\big(z_{\text{img}},\; \text{step}(z_t[:,:,1{:}],\; t),\; \text{dim}=F\big)$$

The image latent is computed *once* before the loop and reused at every step - no per-step re-encoding is needed. The channel dimension $C = 128$ is unchanged relative to T2V, so the existing transformer IR is fully compatible.

3.2.2 Strength Parameter

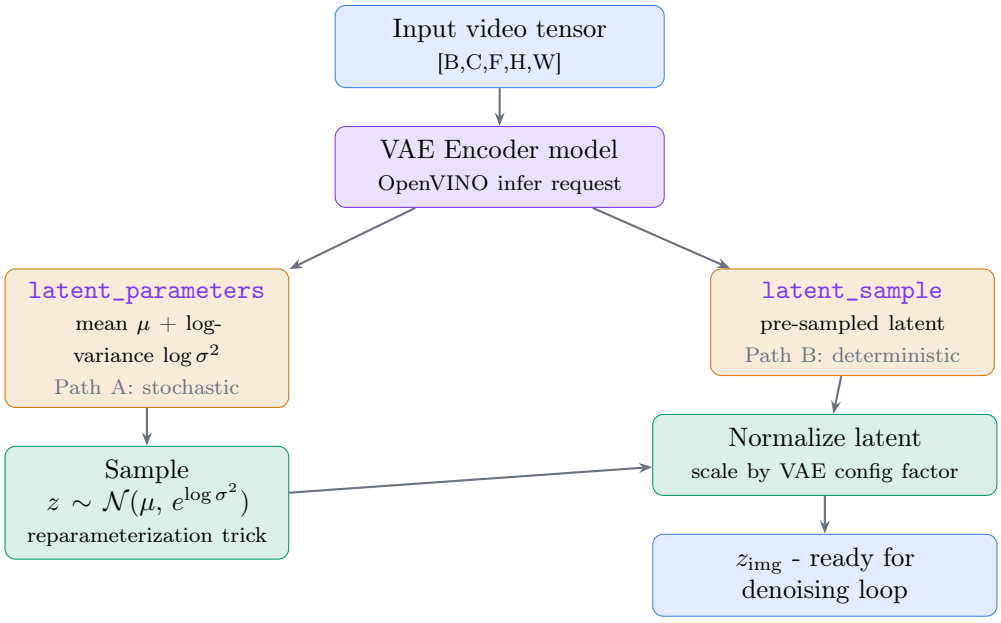
The `strength` parameter $s \in [0, 1]$ controls how many denoising steps are executed, and therefore how strongly the final output is steered by the text prompt versus anchored to the input image structure:

$$t_{\text{start}} = \lfloor s \cdot T \rfloor$$

`strength`	t_{start} (T=50)	Effect
`1.0`	50 (full denoising)	Full denoising from noise; frame 0 anchors composition, text prompt steers the generated frames
`0.7`	35	Balanced - image composition and structure preserved, text steers style and fine detail
`0.5`	25	Strong image fidelity; text has limited influence over composition
`0.3`	15	Output closely follows the conditioning image; minimal generative deviation

3.3 VAE Encoder: Dual Output Handling

The `AutoencoderKLLTXVideo::encode()` implementation handles two possible output formats from the VAE encoder model. Rather than assuming a fixed output format, the implementation inspects the output tensor names at runtime and routes accordingly:



Path A (`latent_parameters`): the model outputs distribution parameters (mean μ and log-variance); the implementation samples z via the reparameterization trick using the provided generator for reproducibility. Path B (`latent_sample`): the model outputs the latent directly with no sampling needed. Both paths normalize the result by the VAE scaling factor before returning.

3.4 API Design

3.4.1 C++ and Python Interface

The `generate()` method gains an optional image overload in both languages. When `image` is absent, the existing T2V code path is taken entirely unchanged. No branching overhead, no behavioral difference for existing users.

C++ API

```
// Text-to-Video (unchanged)
pipeline.generate(prompt, config);

// Image-to-Video (new overload)
pipeline.generate(image_tensor, prompt, config);
```

Python API

```
# Text-to-Video (unchanged)
pipeline.generate(prompt, config)

# Image-to-Video (new overload)
pipeline.generate(image, prompt, config)
```

3.4.2 Configuration Changes

Two fields are added to `VideoGenerationConfig`. The `strength` field is user-facing; `conditioning_image` is set internally by the pipeline and is not intended to be set directly by callers.

Field	Default	Description
`float strength`	`1.0f`	Controls start timestep via $t_{\text{start}} = \lfloor s \cdot T \rfloor$; lower values anchor output more strongly to the conditioning image
`std::optional<Tensor>` `conditioning_image`	`std::nullopt`	Populated internally after `encode()`; exposed in config for cross-language serialization and testing, not for direct user assignment

3.5 Internal Pipeline Execution Flow

The numbered sequence below shows what `Text2VideoPipeline::generate()` does when an image argument is provided, and which steps are new versus unchanged from the existing T2V path. Steps highlighted in purple are the I2V additions.

1. **[I2V]** Encode conditioning image - `m_vae->encode(image, generator)`; cache result as `z_img`
2. Encode text prompt - tokenize and embed via T5 text encoder *(unchanged)*
3. Sample noise `z_noise` for frames $1 \ldots F-1$; compute `t_start` from
4. **[I2V]** Pack image latent via `pack_latents()`; modify `prepare_latents()` to blend with noise in `(B,S,D)` space using
5. **[I2V]** At each step t: scheduler updates non-image tokens only; re-apply image token mask in `(B,S,D)` space before next step
6. Decode final latent `z_0` via `m_vae->decode()` to pixel frames *(unchanged)*
7. Return `ImageGenerationData` with output frames *(unchanged)*

3.6 Validation Strategy

HuggingFace Diffusers' `LTXImageToVideoPipeline` is the numerical reference implementation. For a fixed seed, text prompt, and conditioning image, the OpenVINO GenAI I2V pipeline must produce latents within the tolerances below. CPU is the primary validation target; GPU tolerances are wider due to OpenCL stochastic path differences. C++/Python parity uses the strictest tolerance because both run the same compiled model on the same backend. Any divergence at that level indicates a binding-layer bug, not numerical noise.

Criterion	Threshold	Rationale
CPU vs. Diffusers (absolute)	`atol = 1e-4`	Matches existing cross-backend tolerance in the OpenVINO Keras test suite
CPU vs. Diffusers (relative)	`rtol = 1e-3`	Accounts for FP32 accumulation differences across execution paths
GPU vs. CPU reference	`atol = 1e-3`	Wider window for OpenCL backend; stochastic encode path diverges from CPU
C++ vs. Python parity	`atol = 1e-5`	Strictest. Same model, same backend; any larger divergence is a bug

3.7 Known Technical Challenges

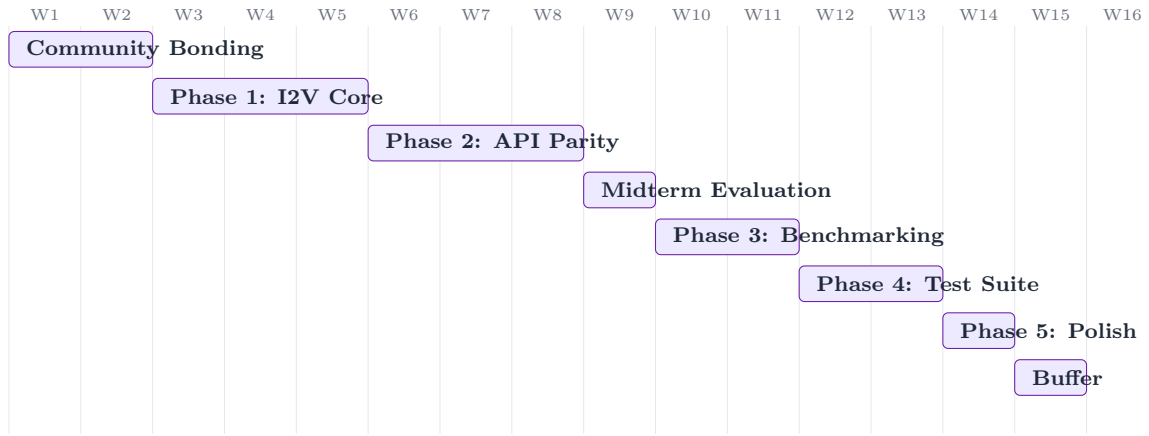
1. **Frame axis conditioning in packed latent space.** The Diffusers implementation applies the conditioning mask in unpacked `(B,C,F,H,W)` space, where frame 0 is a contiguous slice. In OpenVINO GenAI, `prepare_latents()` immediately calls `pack_latents()`, which merges F, H, W into a single sequence dimension S, giving `(B,S,D)`. The latent stays in this packed form for the entire denoising loop - the transformer, scheduler, and re-concatenation all operate on `(B,S,D)`. Frame 0 in unpacked space corresponds to token indices $0 \ldots \frac{H}{p} \cdot \frac{W}{p} - 1$ in S. The implementation must pack the image latent separately via `pack_latents()`, identify the correct token index range for frame 0, and apply the conditioning mask in sequence space throughout the loop. This is the most technically involved part of the project and is allocated the full first week of Phase 1 for design and validation.
2. **Noise scheduling alignment with Diffusers reference.** LTX uses flow-matching schedulers rather than standard DDPM. `FlowMatchEulerDiscreteScheduler::set_timesteps()` already accepts a `strength` parameter and stores it as `m_strength`, and the scheduler already has `set_begin_index()` and `m_begin_index` infrastructure. However, `strength` is currently stored but never used to calculate `begin_index` - that wiring is missing. The GSoC work is implementing the correct slicing logic: `begin_index = num_steps - int(num_steps * strength)`, matching Diffusers' `LTXConditionPipeline.get_timesteps()` exactly, then calling `set_begin_index()` so the scheduler starts at the right offset. This is a targeted addition, not a rebuild.
3. **GPU stochastic encode path and determinism.** The `latent_parameters` path (Path A) uses the reparameterization trick, which produces different results on Intel GPU via OpenCL compared to CPU even with the same seed. For GPU users where output consistency matters, Path B (`latent_sample`) which uses the latent directly with no sampling, is the natural default since it is fully deterministic. The implementation will document this clearly: Path A is used when the model produces distribution parameters; Path B when it produces the latent directly. GPU validation will use Path B as the reference path, with a separate wider-tolerance baseline for Path A if needed.
4. **VAE encoder memory footprint.** After `encode()` runs, z_{img} is cached and the VAE encoder is no longer needed until `decode()` at the end. The question is whether to keep the encoder compiled in memory alongside the transformer and text encoder, or unload it to free VRAM between encode and decode. The existing `Text2VideoPipeline` keeps all models loaded; the I2V path will follow the same strategy by default to avoid recompilation overhead. If memory

pressure becomes a concern during GPU testing, an explicit `unload_vae_encoder()` option can be added. But this is a post-MVP refinement, not a blocker.

5. **API binary compatibility.** The new `generate(image, prompt, config)` overload must not break binary compatibility for existing third-party integrations that call `generate(prompt, config)`. Using a new overload rather than a default argument avoids ABI issues. The `VideoGenerationConfig` additions (`strength` with default `1.0f` and `conditioning_image` with default `std::nullopt`) are backward-compatible by construction. Existing callers that do not set these fields get T2V behavior unchanged.
6. **LoRA adapter state through model reshapes.** `rebuild_models()` must forward `m_compile_properties` so that LoRA adapters survive resolution changes. This was fixed for the T2V path in PR #3362. The I2V code path must not introduce any additional reshape calls that bypass the property-forwarding mechanism. This will be explicitly verified in the test suite.
7. **Python/C++ numerical consistency.** The same compiled model running via Python bindings and C++ must produce outputs within `atol=1e-5` for the same seed and inputs. Any larger divergence indicates a bug in the pybind11 binding layer rather than acceptable numerical noise, and must be investigated and documented before the final evaluation.

4 Project Timeline

4.1 Phase Overview



4.2 Detailed Weekly Plan

Phase / Week	Activities & Deliverables
Community Bonding Weeks 1–2 (May 8–21)	■ Align with mentors on I2V architecture, API signature, and implementation approach ■ Study Diffusers `LTXImageToVideoPipeline` conditioning logic and noise scheduling in depth ■ Set up I2V validation environment; prepare fixed-seed test images and prompts ■ Confirm `generate(image, prompt, config)` API signature with mentors

Phase / Week	Activities & Deliverables
Phase 1: I2V Core Weeks 3–5 (May 22 – Jun 5)	■ Implement VAE encoder (`compile()`, `reshape()`, `encode()`) for `AutoencoderKLLTXVideo` ■ Extend `Text2VideoPipeline::generate()` with optional image conditioning ■ Implement `prepare_latents()` modification with frame-axis conditioning in packed `(B,S,D)` space ■ Implement `strength` parameter and `begin_index` calculation in the scheduler ■ Validate denoising output against Diffusers reference (CPU, fixed seed)
Phase 2: API Parity Weeks 6–8 (Jun 6 – Jun 20)	■ Full C++ API: `generate(image, prompt, config)` overload with type handling ■ Full Python API: matching signature, updated `.pyi` stub ■ C++ sample: `image_to_video.cpp` with CLI arguments ■ Python sample: `image_to_video.py` with equivalent functionality ■ README updates: usage, model requirements, expected outputs
Midterm Week 9 (Jun 21–28)	■ Submit midterm evaluation PR: working I2V pipeline, both language samples, CPU-validated outputs ■ 5-minute demo: input image + text prompt → generated video ■ Draft user-facing documentation covering API and configuration
Phase 3: Benchmarking Weeks 10–11 (Jun 29 – Jul 18)	■ WWB: add I2V + LoRA scenario measuring encode latency, denoising throughput (ms/step), decode latency, end-to-end latency; update WWB README ■ LLM Benchmarking: add I2V pipeline scenario with throughput metrics ■ GPU validation: verify encode, denoising loop, decode on Intel GPU via OpenCL ■ Document CPU and GPU baseline throughput/latency figures
Phase 4: Test Suite Weeks 12–13	

Phase / Week	Activities & Deliverables
(Jul 19 – Aug 8)	■ Cross-language consistency tests: C++ and Python produce identical latents per seed ■ Enforce tolerance: `atol=1e-4, rtol=1e-3` ■ Integration tests using lightweight test model (no full LTX weights in CI) ■ Edge cases: `strength` $\in \{0.0, 0.5, 1.0\}$, invalid inputs, batch/resolution changes ■ Verify LoRA adapter state survives resolution changes in I2V path
Phase 5: Polish Weeks 14–15 (Aug 9 – Aug 25)	■ Full Doxygen comments on all new C++ interfaces ■ Update CODEOWNERS and CI configurations for new test files ■ Final numerical validation pass against Diffusers reference ■ Final mentor review session; address remaining feedback ■ Code cleanup; consistent style across all new files
Buffer Week 16 (Aug 26 – Sep 8)	■ Address critical mentor feedback ■ Submit final evaluation materials ■ Document open issues and future work for post-GSoC contributors

5 Previous Contributions

5.1 OpenVINO GenAI

All contributions below are under the direct review of the project mentors, Anna Likholat (**@likholat**) and Stanislav Gonorovskii (**@sgonorov**). They are listed in the order they were undertaken.

5.1.1 PR #3204 - GGUF LoRA Adapter Support MERGED

Link: github.com/openvinotoolkit/openvino.genai/pull/3204

This was my first contribution to OpenVINO GenAI, submitted in mid-January 2026 and merged on February 4, 2026. At the time, the `Adapter` class in `src/cpp/src/lora/adapter.cpp` only supported safetensors format. I implemented the full GGUF loading path:

- Added `GGUFAdapterImpl` class, mirroring the design of the existing `SafetensorsAdapterImpl` for consistency
- Implemented `convert_gguf_name_to_hf()` to map GGUF tensor naming conventions (`blk.N.attn_q`, `ffn_gate`, etc.) to HuggingFace/OpenVINO naming, using safe literal string matching rather than regex to avoid metacharacter issues
- Modified the `Adapter` constructor to detect `.gguf` extension and route to the appropriate implementation
- Added `ENABLE_GGUF` preprocessor guards for conditional compilation
- Python pybind11 bindings and a 9-test Python suite (2 fast, 7 nightly) covering GGUF parsing, name conversion, alpha scaling, and error handling

Reviewed and merged by @likholat, @rkazants, and @Wovchena

5.1.2 PR #3362 - LoRA Adapter Support for Text2VideoPipeline MERGED

Link: github.com/openvinotoolkit/openvino.genai/pull/3362

Submitted in late January 2026. The issue had been assigned to another contributor who opened a PR but abandoned it shortly after. The initial attempt had minimal baseline code and multiple CI failures. I took over, identified the root causes, and submitted a complete implementation from scratch. The fix required several changes:

- Removed the blocking assertion in `LTXVideoTransformer3DModel::compile()` that was preventing `AdapterController` initialization
- Added `m_transformer->set_adapters()` before the denoising loop, with a null-check to prevent segfaults on uninitialized controllers
- Fixed `rebuild_models()` to forward `m_compile_properties`, ensuring adapters survive model reshapes (e.g. resolution changes)
- Added `std::optional<AdapterConfig> adapters` to `VideoGenerationConfig`
- Exposed `LTXVideoTransformer3DModel::set_adapters()` via pybind11 with updated `.pyi` stub
- Added C++ sample (`lora_text2video.cpp`) and Python sample (`lora_text2video.py`) with README documentation
- Test suite covering constructor, `generate()`, and `set_adapters()` with `AdapterConfig`

Merged by @likholat and @sgonorov

5.1.3 PR #3431 - VAE Encoder for AutoencoderKLLTXVideo DRAFT PR

Link: github.com/openvinotoolkit/openvino.genai/pull/3431

Submitted in late February 2026. While auditing the codebase to scope this GSoC project, I found that both `compile()` and `reshape()` in `AutoencoderKLLTXVideo` contained `// TODO: for img2video` comments. I implemented the full encode path and submitted the PR, but subsequently learned from the mentors that this is part of the GSoC project scope. The PR has been converted to draft accordingly.

- Implemented `compile()` and `reshape()` for the encoder model
- Implemented `encode(video, generator)` taking a `[B,C,F,H,W]` video tensor, running the encoder model, and returning a normalized latent ready for the diffusion transformer
- Handles both model output variants: `latent_parameters` (samples z from the predicted distribution via the reparameterization trick) and `latent_sample` (uses output directly when no sampling is needed)
- Validated the encode-decode pipeline on a real 512×512 image on CPU to confirm the implementation is functionally correct
- Python bindings via pybind11
- 6 tests covering construction, error messages, output shape, determinism with same seed, and variation across seeds

Under review by @likholat and @sgonorov

5.1.4 PR #3546 - WWB LoRA Support for Video Generation OPEN

Link: github.com/openvinotoolkit/openvino.genai/pull/3546

This PR extends the Who What Benchmark (WWB) evaluation tool to support LoRA adapter evaluation for the Video Generation pipeline, matching the existing LoRA workflows already in place for LLM and Image Generation pipelines.

The work involves:

- Adding LoRA adapter options to WWB video generation, consistent in style with the existing LLM and Image Generation pipeline evaluation paths
- Validating against community LoRA adapters for LTX-Video available on HuggingFace (Lightricks/LTX-Video adapters)
- Updating the WWB README with video generation and LoRA usage documentation

5.2 OpenVINO Core Repository

These PRs span multiple components of the OpenVINO codebase, submitted between January and March 2026.

Category	PR		Description
Python API	#34782	MERGED	Fix `opset16` silently omitting 10 `opset15` re-exports.
CPU Plugin	#34639	MERGED	Fix data corruption in JIT eltwise kernel for i8/u8 bitwise ops with broadcast.
CPU Plugin	#34810	MERGED	Fix data corruption: `ReduceMin` after `Concat(Parameter, Constant)` due to in-place memory aliasing in `Edge::init()`.
ONNX FE	#34005	OPEN	Add Relu type constraint test coverage across opsets 6, 13, 14.
PyTorch FE	#33768	OPEN	Add support for `aten::_sparse_mm` and `aten::sparse_coo_tensor`.
PyTorch FE	#33772	OPEN	Add support for `prim::CallMethod` (TorchScript method dispatch).
PyTorch FE	#33778	OPEN	Add support for `aten::Delete`, `aten::str`, `aten::take`.

5.3 Keras 3 OpenVINO Backend

Roadmap Issue: openvinotoolkit/openvino #34017 **All PRs:** keras-team/keras, author:goyaladitya05

This is the largest single body of work in this contribution record. After finding that all previously filed maintainer issues for the OpenVINO backend had already been implemented by others, I audited the entire backend source myself (`keras/src/backend/openvino/`), identified all operator coverage gaps, and obtained permission from Roman Kazantsev **@rkazants** (OpenVINO maintainer) to proceed with parity initiative. I filed Issue #34017 as a structured roadmap, spawning 50+ child issues, and then executed the roadmap personally.

As of March 22, 2026, complete parity is scheduled for April 20, 2026, well before GSoC begins. This work does not overlap with the I2V project scope. It is worth noting that not everything can be implemented: OpenVINO is an inference-only backend, so operations requiring training-time stochastic behaviour (e.g. certain random distribution samplers) are handled with principled `NotImplementedError` and documentation of the limitation.

During this work, several cross-backend bugs were discovered and fixed:

- **Random seed integer overflow** - fixed in both OpenVINO and PyTorch backends (PR #22293)
- **int8/uint8 bitwise ops broadcasting bug** - traced from the Keras Python layer all the way to the JIT eltwise kernel in openvino core, fixed at both layers (PR #22389)
- **Sharded and cross-dtype float weight loading** - fixed silent model loading errors for large sharded checkpoints (PR #22375)
- **Flaky** `test_dropout` - stabilised across backends (PR #22312)

The full scope of implemented operators across all modules is summarised below:

Module	Operators / Work Completed
numpy.py	exp2, trunc, heaviside, var/std, quantile, cumprod, inner, diagflat, searchsorted, einsum, concatenate, unravel_index, nanmax/nanmean/nanmin/nanprod, nanstd, nancumsum, real/imag/isreal, bitwise shifts, add/subtract/multiply dtype promotion fixes, matmul int8 dtype promotion, array/maximum/minimum/reshape/split stabilisation
math.py	Full Fourier Transform suite (fft, fft2, ifft2, rfft, irfft, stft, istft), segment_sum/segment_max, extract_sequences, erfinv
linalg.py	inv, solve, cholesky_inverse
nn.py	conv_transpose, fold, adaptive_average_pool, adaptive_max_pool, moments, binary_crossentropy, softmax, log_softmax
rnn.py	lstm, gru, RNN shape bug fix
core.py	cond, scan, associative_scan, map, switch, fori_loop, vectorized_map, fmod, sinc
trainer.py	evaluate, test_step, test_on_batch

6 General Discussion

6.1 Programming Experience

Python is my primary language - I have been using it for five years across ML research, systems work, and open-source contributions. On the C++ side, I have been writing it for three years. My practical C++ experience comes almost entirely from the OpenVINO codebase: PR #3204 involved implementing `GGUFAdapterImpl` and `convert_gguf_name_to_hf()` in `src/cpp/src/lora/adapter.cpp`; PR #3362 required tracing assertion failures in `LTXVideoTransformer3DModel::compile()`, fixing `rebuild_models()` to forward `m_compile_properties`, and writing C++ samples with pybind11 bindings; PR #3431 implements the full `encode()` path including the `DiagonalGaussianDistribution` class and latent normalization. The CPU plugin PRs in core OpenVINO (#34639, #34810) involved tracing memory aliasing bugs in the JIT eltwise kernel and `Edge::init()` - both required reading and modifying low-level C++ inference engine code.

6.2 How I Know OpenVINO

I first encountered OpenVINO through a senior contributor whose GitHub profile I was browsing - he had several PRs in the repository. That led me to explore the codebase in November 2025, initially out of curiosity about running generative AI models efficiently on local Intel hardware. Within a few weeks I had picked up my first issue and submitted a PR. The Keras 3 backend parity work followed naturally after I noticed the operator coverage gaps there. As of March 2026, I have contributed across openvino.genai, the core OpenVINO repository (Python API, CPU plugin, ONNX and PyTorch frontends), and the Keras 3 OpenVINO backend.

6.3 Why am I a Good Fit?

I have been working in this repository for a few months now. LoRA adapter support, the VAE encoder work, and the WWB benchmarking PR are all either merged or under review by the project mentors. I have also gone through the remaining work in detail: the frame-axis conditioning mechanism in packed latent space, the scheduler strength wiring, full C++/Python API

parity, GPU validation, and a cross-language test suite. The scope is clear and the open questions are documented in the technical challenges section.

Beyond the code, both mentors - Anna Likholat (**@likholat**) and Stanislav Gonorovskii (**@sgonorov**) - have reviewed my work across multiple PRs.

6.4 Career Plans

I am a pre-final year student and will be graduating in 2027. After graduation, I intend to work in industry for a while, likely as a Machine Learning or Deep Learning Engineer, before pursuing a Master's Degree in a related field. The project fits naturally into this direction: working at the C++ implementation level of a production inference framework is exactly the kind of experience I want to build before graduate study. I've learned more in three months of open-source work than I'd have learned in two years of coursework.

I also have an active research interest and am hoping to publish at a top venue before completing my Bachelor's.

6.5 Other Summer Plans

I do not plan to take up an internship this summer. Alongside GSoC, I will be spending some time finishing the writing for my federated learning research, with the goal of submitting to a conference before the year ends. Being a huge football fan, I will also be following the FIFA World Cup 2026.

7 Additional Resources

- **All contributions:** github.com/goyaladitya05
- **Diffusers reference implementation:** `LTXImageToVideoPipeline`
- **Keras 3 parity roadmap:** OpenVINO #34017